

# Password Strength Checker + Generator

How it works and what it outputs

---

## 1. What This Project Does

This is a command-line program with two tools in one. It can check how strong a password is and give tips to improve it, and it can generate a new random strong password of any length you choose.

### Concepts used:

- **Strings** — checking each character of the password.
- **Loops** — going through every character to check its type.
- **Functions** — one function for checking, one for generating.
- **if / elif / else** — deciding the strength label and menu choice.
- **Lists** — collecting tips, and building up the generated password.
- **The random module** — picking random characters and shuffling them.

## 2. How the Program Runs

- The program shows a menu with 3 options in a loop that repeats until you choose Exit.
- Option 1 checks a password you type and shows its strength.
- Option 2 generates a brand-new strong password of a length you choose.
- Option 3 ends the program.

```
===== PASSWORD TOOL =====
1. Check a password
2. Generate a strong password
3. Exit
Choose an option (1-3):
```

## 3. How Password Checking Works

The password is tested against 5 rules. Each rule it passes adds 1 point, so the final score is out of 5:

- Is it at least 8 characters long?
- Does it contain a lowercase letter?
- Does it contain an uppercase letter?
- Does it contain a number?
- Does it contain a symbol (like ! @ # \$)?

If the password is one of a few very common ones (like 123456 or password), the score is forced to 0 no matter what, because these are the first passwords an attacker would try.

The score is then turned into a label:

| Score | Label     |
|-------|-----------|
| 0 - 1 | Very Weak |
| 2     | Weak      |
| 3     | Medium    |

|   |             |
|---|-------------|
| 4 | Strong      |
| 5 | Very Strong |

### Sample run — a weak password:

Enter a password to check: abc

Strength: Very Weak (1/5)

Tips to improve:

- Use at least 8 characters.
- Add at least one uppercase letter.
- Add at least one number.
- Add at least one symbol (like ! @ # \$).

### Sample run — a strong password:

Enter a password to check: Hello123!

Strength: Very Strong (5/5)

Great password!

## 4. How Password Generation Works

The generator builds a password piece by piece so it is guaranteed to be strong:

- It first picks one random lowercase letter, one uppercase letter, one digit, and one symbol — so all 4 types are guaranteed to be present.
- It then keeps adding random characters from all types combined, until the password reaches the length you asked for.
- Finally, it shuffles all the characters using `random.shuffle()`, so the guaranteed characters aren't always in the same position at the start.

### Sample run:

How long should the password be? (min 6): 12

Generated password: gSog%BzQ16l0

Strength: Very Strong (5/5)

If you type a length smaller than 6, the program automatically uses 6 instead, so the password is never too short to be safe.

## 5. A Complete Sample Session

```
===== PASSWORD TOOL =====
```

1. Check a password
2. Generate a strong password
3. Exit

Choose an option (1-3): 1

Enter a password to check: 123456

Strength: Very Weak (0/5)

Tips to improve:

- This password is too common. Choose something unique.

Choose an option (1-3): 2

How long should the password be? (min 6): 10

Generated password: !7RfS9kLp2

Strength: Very Strong (5/5)

Choose an option (1-3): 3

Goodbye!

## 6. How to Run This Program

- Save the file as **password\_tool.py**
- Open a terminal in the same folder
- Type: `python password_tool.py`
- No extra libraries needed — only Python's built-in **random** and **string** modules are used.

---

Password Strength Checker + Generator